

Context window, sessions, and why they are not the same as usage limits

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 3 of 12 · English notes

Inside Claude Code, Vijay spent a long stretch on `/context` because students were adding the Chat **usage percentage** to the **free space in the context window** and panicking when the numbers did not sum. They are different meters. Confusing them will cost you both money and exam items.

Context window: what one request may carry

The context window is the cap on how much text (tokens) the *client can send to the model in a turn*, including system prompt, tools, skills, history, and your new message. In class, default models sat on a **~200k-class** window (he kept saying 272K); some Sonnet SKUs offered **1M** via `/model` (e.g. “Sonnet 1 million”). Always re-read the live models page. A 100 MB dump will not all fit; the product loads what it can.

`/context` showed a 1M-capable session with ~967K still addressable, already partly eaten by system prompt, tools, skills, messages, and **auto-compact buffer**. Free space (~893K in the demo) is what you can still push. The 4% / 39.8K “usage” chip on the same UI was *not* meant to plus-equal that free space to 967K: some of the remainder is conversation already in the thread, rounding, and other buckets. Vyomkesh was right that naïve addition failed; Vijay’s correction was “those are not the same counter.”

Sessions fill the window

Every user/assistant pair stays in the client and is resent (stateless server). Five slash commands already occupied space. Skills you pin occupy space every turn unless you configure them out. Long threads **shrink free space**. That is why session 1 said “open a new chat” and why Code offers `/compact`: summarise history into a shorter memory so later turns fit. If there is not enough dialogue, compact no-ops (“not enough messages”).

Auto-compact

`/config auto compact = true` (classroom syntax) tells Code to compact when you hit the window, then keep going. The setting is written into Claude’s JSON config (see next chapter for user / project / local files). Auto-compact is a *context* strategy. It does not refill your Team usage bar.

Usage limits: the other meter

Cowork/Chat Settings → Usage is a **rate/quota window** (five-hour reset + weekly cap in the 5 August sketch; enterprise caps are admin-set). It measures how hard you have hit the service lately, not how many tokens remain in the current prompt. API rate limits are a third plane ([docs](#)). You cannot `/config` a hard “only

spend 2k tokens per turn” in Code; you steer with prompting and by not stuffing CLAUDE.md with encyclopedias. /context is observability, not a budget governor.

Meter	What it answers	Where you saw it	If it hits the wall
Context window	Can this turn’s payload fit?	/context, model picker 272K vs 1M	Compact, new session, smaller attachments, or a 1M SKU
Product usage bar	Have I exhausted my plan’s rolling allowance?	Desktop Settings → Usage	Wait for reset or ask org for quota
API rate limits / billing	May this key call Messages right now, and at what \$?	Console / API errors	Backoff, Batches, or a different key/ workspace

CLASSROOM TOKEN MATHS

272K, 1M, 967K free, 5-hour 1M illustrations were live-UI numbers on 6 August 2026. Recheck **models** before you quote them in an exam written answer.

Prompt cache (name it, do not mix it)

Session 1 mentioned cached tokens on some stacks; Claude Chat enterprise was “input + output.” Prompt caching as an API feature is a later module. Compact is not cache. Usage reset is not cache. If a stem says “repeated system prompt across 10k jobs,” think cache/Batches, not /compact.

Study drill

Write three sentences you can paste into a scenario: (1) I would start a new Claude Code session because the previous thread had consumed the window. (2) I would not expect Settings → Usage to drop after compact. (3) I would pick a 1M model only if the retrieved corpus truly needs it, because effort and window both cost.

Official documentation

- **Models (context windows)**

<https://docs.claude.com/en/docs/about-claude/models>

Live window sizes per ID.

- **Prompt caching**

<https://docs.claude.com/en/docs/build-with-claude/prompt-caching>

The API feature people confuse with compact and with usage bars.

- **Rate limits**

<https://docs.claude.com/en/api/rate-limits>

API meter, not /context.

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

Where slash commands and sessions are documented now.

- **Workspaces**

<https://docs.claude.com/en/docs/build-with-claude/workspaces>

Project grouping versus a single overflowing thread.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.